

Factory Method

—flipout classroom—

Patrones de Creación

Foco:

- Desacoplar el cliente (el código que necesita un objeto) de:
 - a. las clases concretas que se están creando,
 - b. la lógica/reglas que rigen la creación de objetos relacionados.

Objetivo:

- **Encapsular la Lógica de Creación**
- **Abstraer el Tipo de Objetos**
- **Facilitar la Intercambiabilidad**
- **Promover la Reutilización**

The Sacred Elements of the Faith

the holy
origins

the holy
structures

107 FM Factory Method	the holy behaviors					139 A Adapter		
117 PT Prototype	127 S Singleton				223 CR Chain of Responsibility	163 CP Composite	175 D Decorator	
87 AF Abstract Factory	325 TM Template Method	233 CD Command	273 MD Mediator	293 O Observer	243 IN Interpreter	207 PX Proxy	185 FA Façade	
97 BU Builder	315 SR Strategy	203 MM Memento	305 ST State	257 IT Iterator	331 V Visitor	195 FL Flyweight	151 BR Bridge	

the holy
behaviors

Factory Method

Para poder hablar de construcción de cosas
complejas...

Elementos químicos I

- Elemento

- Átomos: oxígeno(O), hidrogeno (H), Sodio (Na), calcio (Ca), cloro(Cl)
- Uniones: (OH)⁻, H₂O, NaCl, CaO, NaOH

Símbolo	Peso	Carga	Clasificación
H	1	+1	No metal
O	16	-2	No metal
Cl	35	-1	No metal
Na	23	+1	Metal
Ca	40	+2	Metal

Reglas	Ejemplo
● Metal + No metal → permitido ● No metal + No metal → permitido ● Metal + Metal → no permitido	● ✗ Na + Ca → inválido ● ✓ Na + Cl → válido (NaCl) ● ✓ Ca + O → válido (CaO) ● ✓ H + O → válido (H₂O)



Elementos químicos I

- Elemento

- Átomos: oxígeno(O), hidrogeno (H), Sodio (Na), cloro(Cl)
- Uniones: (OH)⁻, H₂O, NaCl, CaO, NaOH

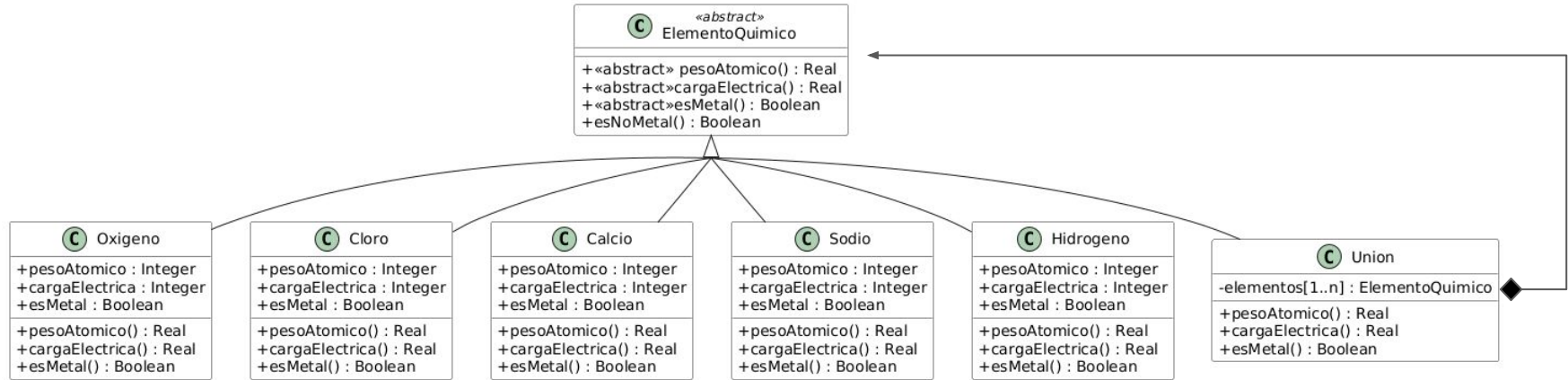
Símbolo	Peso	Carga	Clasificación	Ejemplo
H	1	+1	No metal	• Metal + No metal → permitido • No metal + No metal → permitido • Metal + Metal → no permitido
O	16	-2	No metal	
Cl	35	-1	No metal	
Na	23	+1	Metal	
Ca	40	+2	Metal	

Átomos y Uniones se comportan igual



Elementos químicos II

- Considerando los uniones:
 - La carga total es la suma de las cargas de sus componentes.
 - Si la carga de la union = 0 → molécula
 - Si la carga de la union != 0 → ión
 - El peso molecular total es la suma de los pesos moleculares de sus componentes.



Variable `UnionQuimica.elementos` apunta a superclase
`ElementoQuimico.esNoMetal()` retorna `!this.esMetal()`

Mezclador de Pinturas

Una empresa constructora utiliza un programa para diseñar pinturas

- Considera colores en stock
- Tipo de pintura requerida (tipo de superficie, tiempo de secado, repintado)
- Superficie a cubrir → Costo
- Requerimiento para transporte (inflamable o no inflamable)
 - Depende de la base o diluyente que usa la pintura

Pintura

Latex

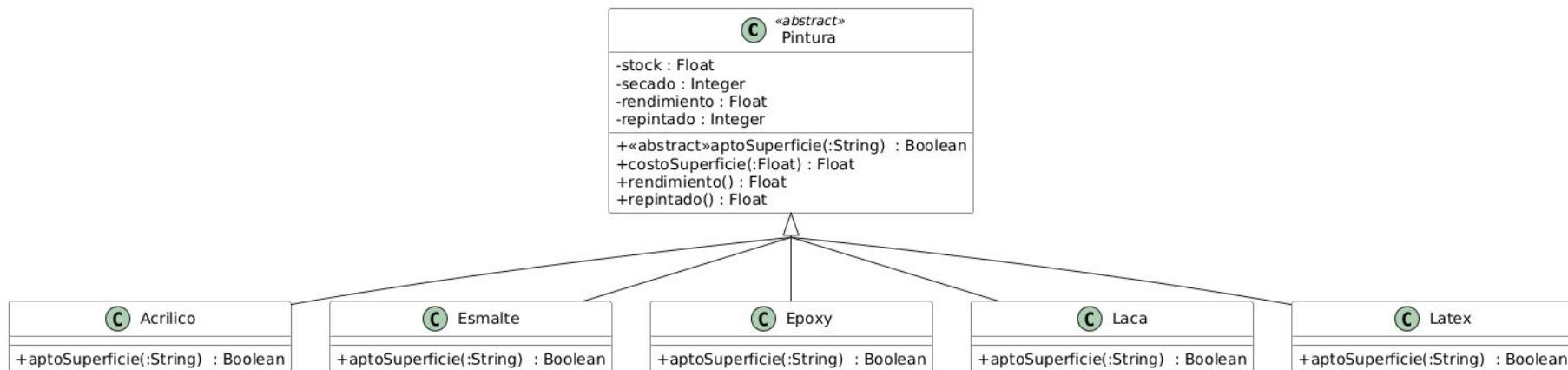
Acrilico

Esmalte

Laca

Epoxy





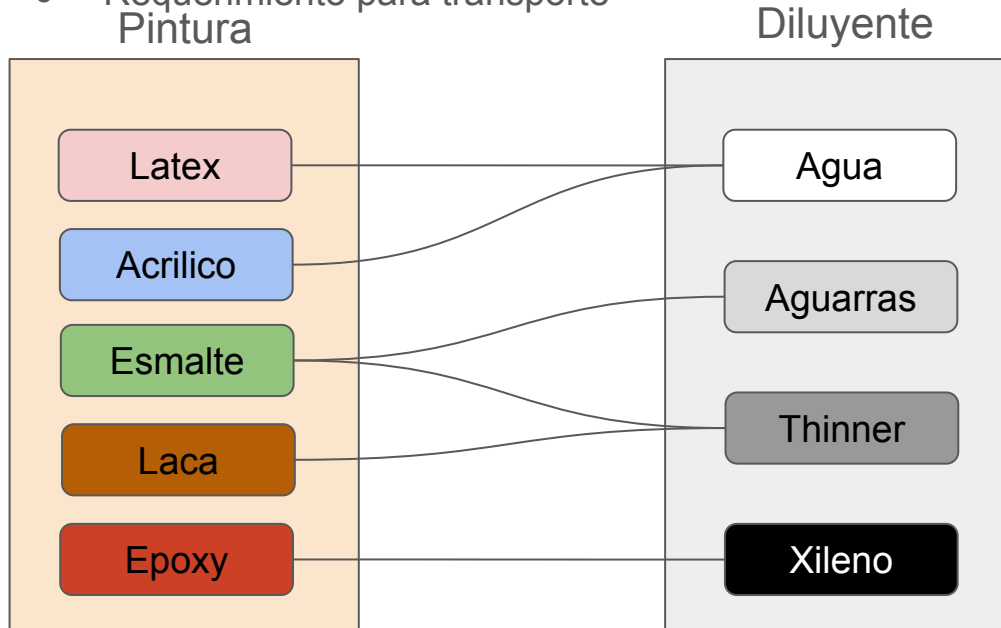
Consideraciones

- El diagrama de clases no incluye constructor que solo setea las variables
- Si la pintura es inflamable o inerte depende de su diluyente
 - Existen pinturas con multiples diluyentes, con diferente indice de inflamabilidad
- El sistema debería permitir agregar con facilidad
 - Nuevos diluyentes
- ⇒ no es buena idea tener la inflamatoriedad de una pintura “hard-coded”
 - Podriamos utilizar el modelo ElementoQuimico para retornar el diluyente
 - Modificar UnionQuimica para que nos diga si es inflamable o no

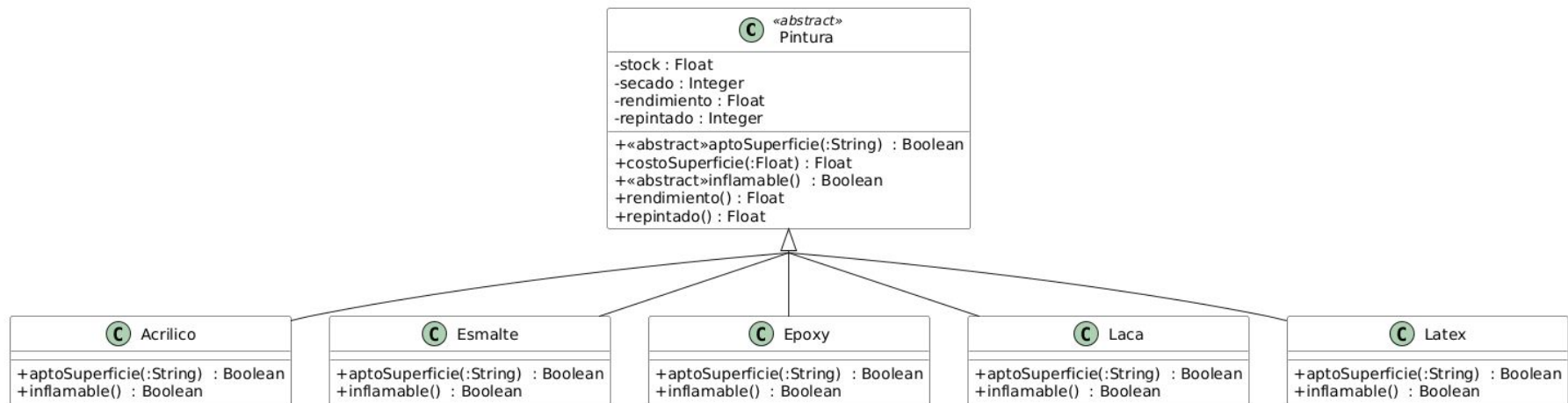
Mezclador de Pinturas

Una empresa constructora utiliza un programa para diseñar pinturas

- Considera colores en stock
- Tipo de pintura requerida (tipo de superficie, tiempo de secado, mantenimiento)
- Superficie a cubrir → Costo
- Requerimiento para transporte



Modelo de Pinturas



```
inflamable() {
    UnionQuimica ion = new UnonQuimica();
    ion add(new Hidrogeno());
    ion add(new Oxigeno());

    UnionQuimica agua = ion.add(new Hidrogeno());

    return agua.esInflamable();
}
```

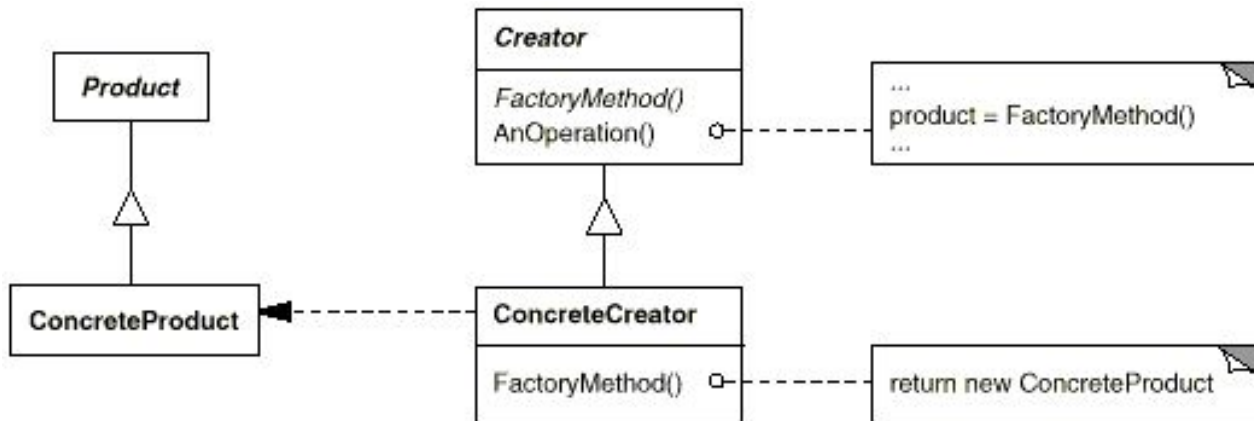
- Cada subclase debe implementar `inflamable()`
 - Esto depende del diluyente de la pintura
- En el código presentado no está el diluyente
 - El diluyente se define con la jerarquía `ElementoQuímico`

```
esInflamable() {
    UnionQuimica agua = new UnonQuimica();
    agua add(new Hidrogeno());
    agua add(new Hidrogeno());
    agua add(new Oxigeno());

    return agua.esInflamable();
}
```

Factory Method

- Intención: Define una “interface” para la creación de objetos, mientras permite que subclasses decidan qué clase se debe instanciar. Separa la instanciación/configuración de un objeto Producto en un método que puede ser extendido por subclasses del creador
- Alias: Virtual Constructor



Factory Method

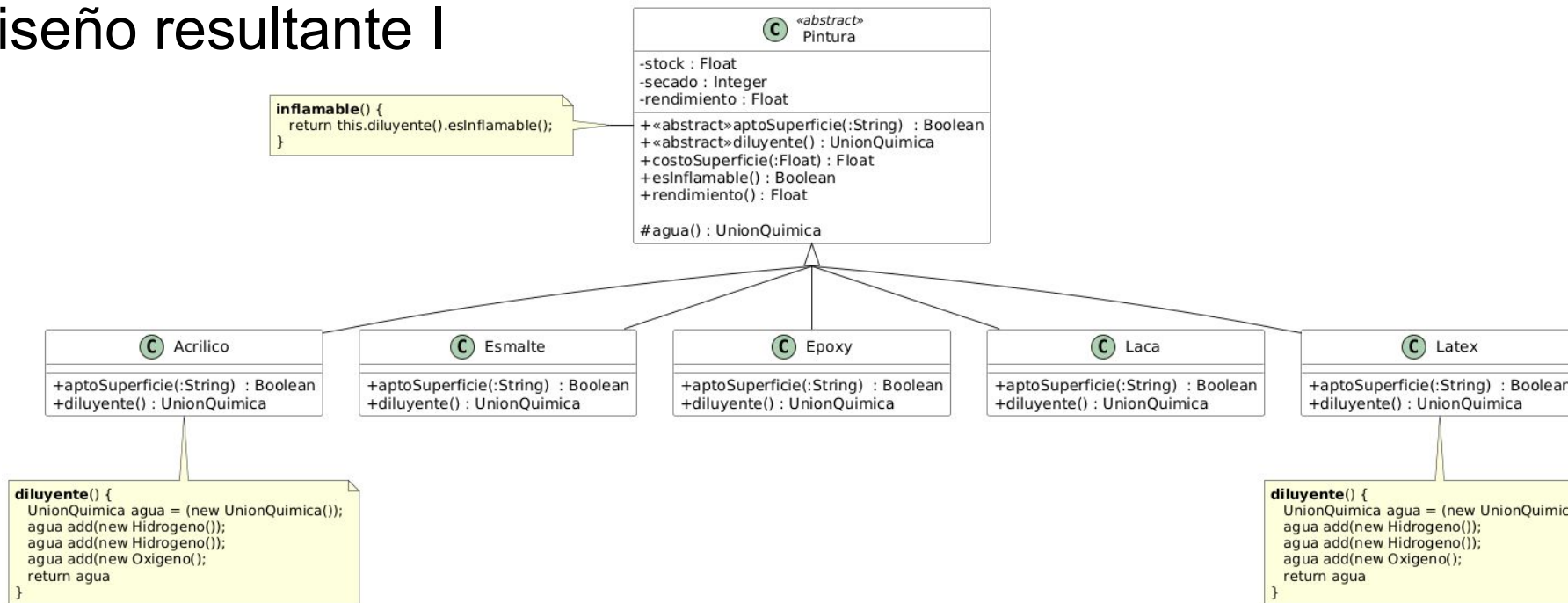
Participantes (Roles)

- **Product**
 - Define la interface of objetos creados por el “factory method”
- **Concrete Product**
 - Implementa la interface definida por el Product
- **Creator**
 - Declara el “factory method” (abstracto o con comportamiento default)
- **ConcreteCreator**
 - Implementa el “factory method”

Consecuencia

- Le da nombre a una configuración particular (virtual constructor)
- La responsabilidad (de construcción) la tiene quien usa el “objeto complejo”

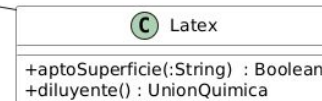
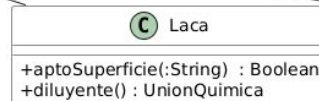
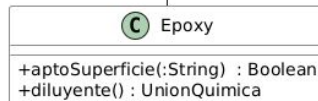
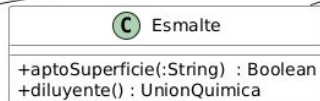
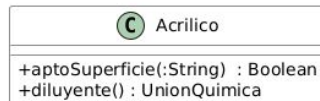
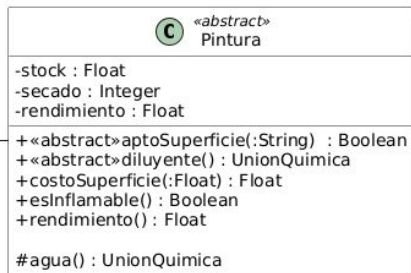
Diseño resultante I



- inflamable() esta definido en la superclase e invoca a diluyente()
- Diluyente debe ser implementado por todas las subclases
 - Acrilico y Latex tienen el mismo diluyente → código repetido
 - Esto no es un problema de FactoryMethod sino una cuestión del dominio, no invalida la solución propuesta por el patrón

Trabajo para repasar

```
inflamable() {  
    return this.diluyente().esInflamable();  
}
```



```
diluyente() {  
    UnionQuimica agua = (new UnionQuimica());  
    agua add(new Hidrogeno());  
    agua add(new Hidrogeno());  
    agua add(new Oxigeno());  
    return agua  
}
```

```
diluyente() {  
    UnionQuimica agua = (new UnionQuimica());  
    agua add(new Hidrogeno());  
    agua add(new Hidrogeno());  
    agua add(new Oxigeno());  
    return agua  
}
```

1. Complete el uso del patrón FactoryMethod en el resto de la jerarquía
2. Agregue el metodo *esInflamable()* en la jerarquía de ElementoQuimico
3. Aplique Refactorings vistos en la materia para eliminar el codigo repetido
 - a. Considere que existe más de una solución
 - b. Evalúe pros y cons de cada una

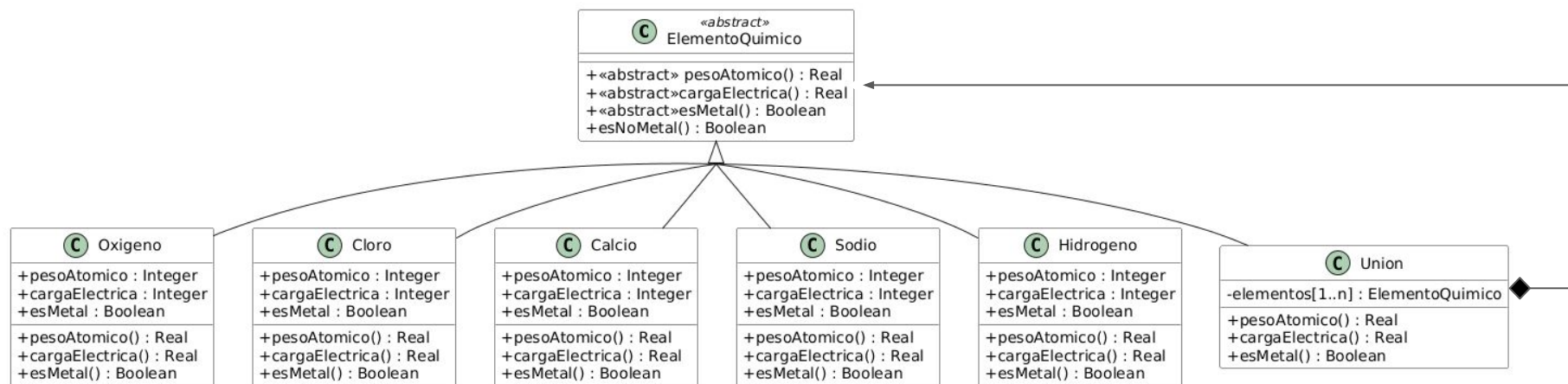
Ahora que entendimos Factory Method...

Factory Method Unleashed



Factory Method Unleashed I

- Las clases hojas
 - son elementos de la Tabla Periodica de Elementos
 - Tiene los mismas variables y métodos “getters” ⇒ variables y código repetido :-)
 - Cada elemento atómico tiene diferentes valores que lo describen
- Por qué no tener una clase única para los elementos atómicos?
 - Quien podría ser responsable de proveer objetos configurados como Oxígeno, Cloro y los otros elementos atómicos

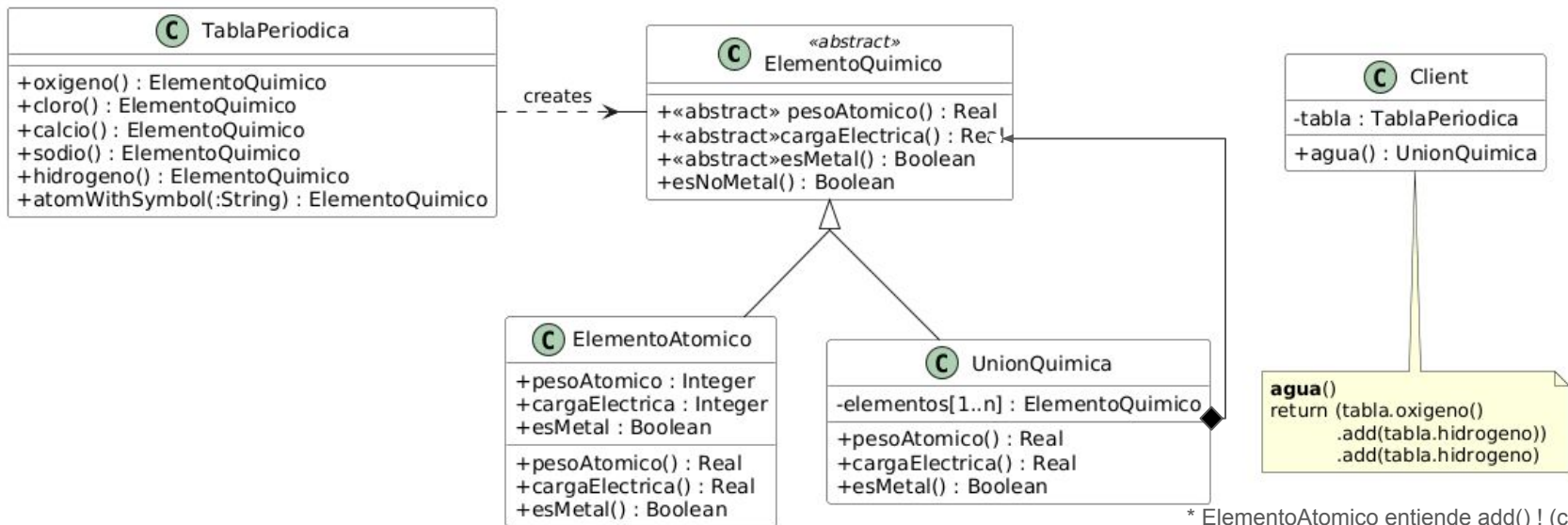


FactoryMethod Unlished II

- Por qué no tener una clase única para los elementos atomicos?
 - Quien podría ser responsable de proveer objetos configurados como Oxígeno, Cloro y los otros elementos atómicos (**responsabilidad == método**)
 - Adonde implementamos el método que configura, por ejemplo, Oxigeno?
 - Podría ser un constructor de ElementoAtómico (subclase de ElementoQuimico)
 - Hay 118 elementos atomicos en la TablaPeriodica
 - Dependiendo del uso la def de Oxigeno puede cambiar (Simulador, Mezclador, etc)
 - Podría ser un “factory method” de alguien sabe todo sobre Elementos Quimicos Atomicos: TablaPeriodica

FactoryMethod aplicado a ElementoQuímico

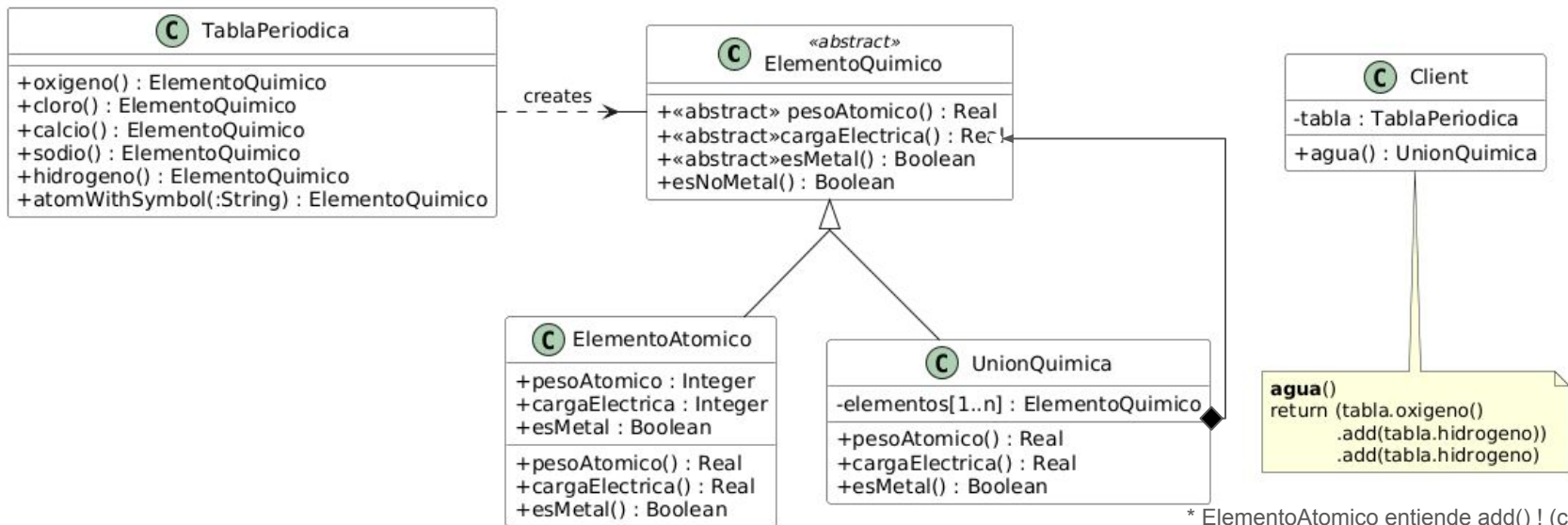
- Permite codificar la lógica de instanciación/configuración de objetos en un unico lugar.
 - Nacio como una manera de mitigar el problema de instanciacion con jerarquías paralelas
 - Tiene un valor importante al poner “nombre” a objetos complejos con reglas de instanciacion “fijas”
 - Agua, Agua Oxigenada, SaldeMesa (cloruro de Sodio)...
 - Combo BigMac, Whopper, CeroVeggie...



* ElementoAtomico entiende add() ! (cómo?)

FactoryMethod in a nutshell

- Permite codificar la lógica de instanciación/configuración de objetos en un unico lugar.
 - Nacio como una manera de mitigar el problema de instanciación con jerarquías paralelas
 - Tiene un valor importante al poner “nombre” a objetos complejos con reglas de instanciacion “fijas”
 - Agua, Agua Oxigenada, SaldeMesa (cloruro de Sodio)...
 - Combo: BigMac, Whopper, CeroVeggie...



* ElementoAtomico entiende add() ! (cómo?)